CLASS REFERENCE

Klassen- und Architekturüberblick

SDN.Leistung LMG — Transaktionssteuerung und Fachschicht auf Java 21 und Oracle

Projekt Leistungsmanagement Grundsicherung

69 Klassen in 12 Paketen, mit Erklärungen und Lernpfad

Stand: 12. Oktober 2026 (Version 0.5.8)

Inhaltsverzeichnis

1 Das Projekt in Kürze 3

1.1 Worum es geht 3
1.2 Die Paketstruktur im Überblick 3
1.3 Wie man dieses Dokument nutzt 4

2 Der Transaktionskern: de.allgemein.tx 5

2.1 Idee und Zusammenspiel 5
2.2 Transaktionsführung (Paketwurzel) 5
2.3 Task-Steuerung (Transaktionsmonitor-Rolle) 7
2.4 Verbindungspools 8
2.5 Das Saga-Paket tx.saga — lange Geschäftsvorgänge 8
2.6 Das Audit-Paket tx.audit 13
2.7 Das Monitoring-Paket tx.monitoring (JMX) 14
2.8 Das Demo-Paket tx.demo 15

3 Die Fachschicht SDN.Leistung: de.allgemein.lmg 16

3.1 Idee und Zusammenspiel 16
3.2 model — die Wertobjekte der Fachwelt 16
3.3 dao — der Datenzugriff 18
3.4 calc — Bedarfsberechnung nach SGB XII 19
3.5 config — externalisierte Konfiguration 20
3.6 service — die Fachdienste 21
3.7 saga — die fachliche End-to-End-Saga 24
3.8 web — Weboberfläche und JSON-API 25

4 Der Weg einer Auszahlung durch alle Schichten 27

5 Empfohlene Lernreihenfolge 29

Hinweis: Dieses Inhaltsverzeichnis wird über Feldfunktionen erzeugt. Nach Änderungen am Dokument bitte mit Rechtsklick auf das Verzeichnis und Felder aktualisieren wählen, damit die Seitenzahlen stimmen.

1 Das Projekt in Kürze

1.1 Worum es geht

Dieses Dokument beschreibt alle 69 Java-Klassen des Projekts **Leistungsmanagement Grundsicherung** (Oberfläche und Fachmodul: **SDN.Leistung**) und erklärt, was jede Klasse leistet. Es richtet sich an Leser, die den Code lernen wollen: pro Klasse gibt es die Rolle im Gefüge, eine verständliche Erklärung der Aufgaben und die wichtigsten Konzepte.

Das Projekt ist eine eigene **Transaktionssteuerung** für Java 21 und Oracle — der Ersatz für einen klassischen TP-Monitor (Transaktionsmonitor) auf dem Großrechner, wie er im Umfeld der Sozialverwaltung teuer lizenziert eingesetzt wird. Das Leitprinzip lautet: **Oracle liefert den transaktionalen Kern** — ACID, Sperrverwaltung, Redo-Logging und Recovery. Wir programmieren nur die Steuerungsschicht darüber: Beginn, Commit und Rollback von Transaktionen, Wiederholung bei Sperrenkonflikten, Zeitlimits, Parallelitätssteuerung und lange Geschäftsvorgänge als Saga.

Der Code besteht aus zwei Hälften: dem **Transaktionskern** im Paket `de.allgemein.tx` (34 Klassen, wiederverwendbar und fachfrei) und der **Fachschicht** im Paket `de.allgemein.lmg` (35 Klassen: Grundsicherung im Alter und bei Erwerbsminderung nach dem SGB XII). Zwei Fachbegriffe ziehen sich durch alles: ein Leistungsberechtigter heißt **SozialFall** (Tabelle `SOZIAL_FALL`), ein geführter Vorgang heißt **Bedarfsgemeinschaft** (Tabelle `BEDARFSGEMEINSCHAFT`).

1.2 Die Paketstruktur im Überblick

Die folgende Tabelle zeigt alle zwölf Pakete mit Klassenzahl und Aufgabe. Die Kapitel 2 und 3 behandeln sie in derselben Reihenfolge.

Tabelle 1: Pakete des Projekts

Paket	Klassen	Aufgabe
de.allgemein.tx	14	Transaktionsführung, Retry, Task-Steuerung, Verbindungspools
de.allgemein.tx.saga	9	Saga-Engine: lange Geschäftsvorgänge als Kette kurzer Transaktionen
de.allgemein.tx.audit	3	Revisionssicheres Audit-Log für Transaktions- und Saga-Ereignisse
de.allgemein.tx.monitoring	6	JMX-Überwachung (MBeans) für TaskController und Pools
de.allgemein.tx.demo	2	Saga-Demo auf Demo-Tabellen (start.bat demo)
de.allgemein.lmg.model	6	Wertobjekte der Fachwelt (SozialFall, Zahlung, Bescheid ...)
de.allgemein.lmg.dao	6	Datenzugriff auf die Oracle-Tabellen der Fachschicht

Paket	Klassen	Aufgabe
de.allgemein.lmg.calc	4	SGB-XII-Bedarfsberechnung, Satztabelle, Bescheid-Snapshot
de.allgemein.lmg.config	1	Externalisierte Konfiguration (Kaskade Datei, Umgebung, System)
de.allgemein.lmg.service	12	Fachdienste: Zahlungen, Bescheide, Kassenlauf, Validierung
de.allgemein.lmg.saga	1	Fachliche End-to-End-Saga: Antrag, Prüfung, Bescheid, Auszahlung
de.allgemein.lmg.web	5	Weboberfläche und JSON-API auf dem JDK-HttpServer

Quer dazu liegen die Ressourcen: die DDL-Skripte in `ddl/` (Oracle-Schema, Fachschema, Seed-Daten), die Tests unter `src/test/java` (unter anderem `SzenarioRegressionTest` mit zehn End-to-End-Fällen) und der Mitarbeiter-Simulator `testsweet/`, der mit N Sachbearbeitern parallel arbeitet. Stand dieser Beschreibung ist Version 0.5.8 vom 12. Oktober 2026.

1.3 Wie man dieses Dokument nutzt

Kapitel 2 erklärt den Transaktionskern, Kapitel 3 die Fachschicht, Kapitel 4 verfolgt einmal einen kompletten Request durch alle Schichten, und Kapitel 5 schlägt eine Lernreihenfolge vor. Wer schnell einsteigen will, liest Kapitel 4 zuerst: Dort tauchen die wichtigsten Klassennamen in ihrem natürlichen Zusammenspiel auf.

Vertiefung bietet die Projektdokumentation: `doc/architektur.md` (Bausteine und Abläufe), `doc/leistungsmanagement.md` (Fachschicht), `doc/regressionstests.md` (die zehn Testfälle mit erwartetem Datenbank-Endzustand) und `doc/rechtsgrundlagen.md` (Rechtsquellen mit Herkunftsnachweisen). Die Testklassen sind außerdem ein hervorragendes Nutzungsbild: Wer sieht, wie eine Klasse aufgeufen und geprüft wird, versteht ihren Vertrag.

2 Der Transaktionskern: de.allgemein.tx

2.1 Idee und Zusammenspiel

Der Kern stellt zwei Ausführungsmodelle bereit. Für kurze Arbeitseinheiten (typisch: eine Maske speichern, eine Zahlung buchen) gibt es `TxManager.execute`: leihe eine Verbindung aus dem Pool, beginne die Transaktion, führe die Arbeit aus, committen oder zurückrollen — und wiederhole nur bei Sperrenkonflikten. Für lange Geschäftsvorgänge, die nicht in einer Transaktion laufen dürfen (Antrag, Bescheid, Auszahlung), gibt es die `SagaEngine`: sie zerlegt den Vorgang in eine Kette kurzer Transaktionen und hält den Fortschritt in Journal-Tabellen fest, sodass nach einem Absturz sauber kompensiert oder fortgesetzt werden kann.

Darum liegen drei Querschnittsdienste: der `TaskController` begrenzt die Parallelität und überwacht Zeitlimits (die Rolle, die beim Vorbild der Transaktionsmonitor selbst spielte), das `TxAuditLog` protokolliert revisionssicher ohne Sozialdaten, und das `TxMonitoring` legt die Kennzahlen über JMX offen. bewusst gilt überall: keine Frameworks, keine verteilten Transaktionen (kein XA) — nur Java 21, JDBC und Oracle.

Für Lernende wichtig: Die Klassen sind bewusst klein und rollenklar gehalten (der ganze Kern umfasst rund 3.400 Zeilen). Fast jede Klasse trägt einen ausführlichen deutschen Klassenkommentar, der die Entscheidungen begründet — dieser hier ist eine Karriere durch den Code.

2.2 Transaktionsführung (Paketwurzel)

`TxManager`

`de.allgemein.tx.TxManager · final class · 149 Zeilen`

Der Kern der Steuerungsschicht. `TxManager` führt eine Arbeitseinheit als Oracle-Transaktion aus: Verbindung aus dem Pool holen, Autocommit aus, Arbeit laufen lassen, dann Commit bei Erfolg oder Rollback bei Fehler. Oracle selbst liefert ACID, Sperrverwaltung und Recovery — gesteuert werden hier nur Beginn, Abschluss, Zeitlimit und Wiederholung.

Zwei Garantien sind besonders: Beim Durchreichen eines `Error` oder `OutOfMemoryError` wird trotzdem zurückgerollt (in JDBC würde `setAutoCommit(true)` sonst implizit committen — genau das darf nie passieren), und wiederholt wird nur, wenn die `RetryPolicy` den Fehler als Sperrenkonflikt einstuft. Neben der bequemen `execute`-API gibt es eine manuelle API (`begin/commit/finish`) für Sonderfälle.

`TxWork`

`de.allgemein.tx.TxWork · interface`

Ein funktionales Interface und der Vertrag des Fachcodes: `TxWork<T>.run(Connection)` enthält die SQL-Arbeit einer kurzen Transaktion und liefert ein Ergebnis vom Typ T. Der Fachcode sieht nur die frische Verbindung — alles andere (Beginn, Commit, Rollback, Retry, Zeitlimit) übernimmt der

`TxManager`. Dieses Interface ist die wichtigste Naht des ganzen Projekts: Jede Fachfunktion der Kapitel 3 ist ein `TxWork`.

TxContext

`de.allgemein.tx.TxContext · final class · AutoCloseable`

Repräsentiert genau eine laufende Oracle-Transaktion (Vorbild: ein CICS-Task mit SYNCPOINT). `TxContext` hält die geholte Verbindung und die Deadline des Zeitlimits; schließen gehört dem Aufrufer über `release` des Pools. DAOs der Fachschicht holen sich über ihn die Verbindung der laufenden Transaktion — dadurch brechen keine Transaktionsgrenzen.

TxOptions

`de.allgemein.tx.TxOptions · record (isolation, timeout, maxAttempts)`

Die drei Stellschrauben einer Transaktion: Isolationsebene, Zeitlimit und maximale Versuchsanzahl. Wichtig ist die Zeitlimit-Semantik: `timeout` gilt pro Versuch — der TxManager setzt die Deadline je Versuch neu. Der Wachhund des TaskController misst dagegen die Gesamtzeit inklusive aller Wiederholungen und Backoff-Pausen. Wer also Wiederholungen erlaubt, wählt das Task-Zeitlimit großzügig (Faustregel: Zeitlimit mindestens Versuchsanzahl mal Einzellaufzeit).

TxStatus

`de.allgemein.tx.TxStatus · enum`

Der Lebenszyklus einer geführten Transaktion als Aufzählung. Ein kleines Teil, aber nützlich zum Lesen der Protokolle: Der Status macht sichtbar, in welcher Phase eine Transaktion war, wenn etwas schiefging.

TxException

`de.allgemein.tx.TxException · RuntimeException, Feld retryable`

Die Fehlerart der Transaktionssteuerung. Das Feld `retryable` sagt, ob ein erneuter Versuch der ganzen Transaktion sinnvoll sein kann (etwa ein Deadlock) oder nicht (etwa eine verletzte Prüfbedingung). Fachfehler erben von dieser Klasse (`FachlicheException` in Kapitel 3), damit sie der TxManager unangetastet durchreicht: Rollback ja, Wiederholung nein.

RetryPolicy

`de.allgemein.tx.RetryPolicy · final class`

Entscheidet, welche Datenbankfehler einen Wiederholversuch rechtfertigen, und federt zwischen den Versuchen zurück (exponentieller Backoff). Wiederholbar sind genau die Sperrenkonflikte: `ORA-00054` (Ressource belegt), `ORA-00060` (Deadlock) und `ORA-08177` (Serialisierbarkeitskonflikt). Alles andere ist fachlich oder technisch endgültig und bricht sofort ab — wiederholtes Rennen gegen denselben Fachfehler wäre Zeitverschwendung.

2.3 Task-Steuerung (Transaktionsmonitor-Rolle)

Die vier Klassen dieses Abschnitts bilden die Transaktionsmonitor-Rolle nach: Arbeit wird als Task eingereicht, die Parallelität ist begrenzt, ein Wachhund überwacht Zeitlimits, und eine Statistik beschreibt den Durchlauf. Jede Web-Anfrage der Fachschicht läuft als solcher Task (siehe `LmgServer`).

`TaskController`

`de.allgemein.tx.TaskController · final class · 266 Zeilen · AutoCloseable`

Die Task-Steuerung im Stil eines Transaktionsmonitors. Der Controller nimmt Arbeit über `submit` entgegen, begrenzt die Zahl paralleler Tasks (beschränkter Thread-Pool mit Warteschlange) und überwacht jede Ausführung mit einem Wachhund: Nach Ablauf des Zeitlimits wird der Task unterbrochen — der Interrupt lässt den laufenden JDBC-Aufruf fehlschlagen, wodurch die Datenbank die Transaktion zurückrollt.

Zwei Feinheiten aus der Praxis: Der Wachhund-Timer startet erst, wenn der Task wirklich zur Ausführung kommt (Wartezeit in der Schlange zählt nicht), und das Zeitlimit gilt für die gesamte Ausführung einschließlich aller Wiederholungsversuche und Pausen des TxManager. Der ehrlich dokumentierte Rest: Trifft der Interrupt den Arbeitsthread genau im Commit-Fenster, kann der Commit durchgehen — dieses winzige Fenster ist Teil der JDBC-Semantik und wird im Kommentar offen benannt.

`TaskHandle`

`de.allgemein.tx.TaskHandle<T> · final class`

Das Handle auf einen eingereichten Task, ähnlich einem `Future`: Ergebnis abholen (blockierend), Aufgabe abbrechen, Status prüfen. Die Webschicht nutzt es, um auf das Ergebnis einer Anfrage zu warten; der TaskController meldet darüber auch den Wachhund-Abbruch.

`TaskInfo`

`de.allgemein.tx.TaskInfo · final class (paketsichtbar)`

Laufzeitinformationen zu einem eingereichten Task (Vorbild: CICS-Task-Anzeige im Monitoring). Die Klasse ist bewusst nur im Paket sichtbar — nach außen reicht die Summe in `TaskStats` und die Auswahl über das JMX-Monitoring.

`TaskStats`

`de.allgemein.tx.TaskStats · record`

Die Momentaufnahme der Task-Statistik: eingereichte, laufende, abgeschlossene und fehlgeschlagene Aufgaben sowie Zeitwerte. Vorbild sind die CICS-Transaktionsstatistiken; angezeigt wird das Ganze im Adminbereich der Weboberfläche und über JMX.

2.4 Verbindungspools

Alle Datenbankzugriffe laufen über das schmale Interface `ConnectionPool`. Genau zwei Implementierungen gehören zum Projekt — der Austausch gegen HikariCP oder Oracle UCP in Produktion wäre ein einziger Konstruktor-Aufruf, weil alles nur an `getConnection/release` hängt.

`ConnectionPool`

`de.allgemein.tx.ConnectionPool · interface · AutoCloseable`

Die Schnittstelle für Verbindungspools der Transaktionssteuerung: Verbindungen entleihen, zurückgeben, Pool schließen. Sie erlaubt den nahtlosen Tausch zwischen dem schlanken `SimpleConnectionPool` für Entwicklung und Demo, dem `UcpConnectionPool` für Produktion und Drittanbieter-Pools. An dieser Stelle greift die Architekturentscheidung, keine Abhängigkeit an einen bestimmten Pool zu schmieden.

`SimpleConnectionPool`

`de.allgemein.tx.SimpleConnectionPool · 251 Zeilen`

Der minimale Pool für Demo und Übergangsphase — bewusst einfach, aber mit hartgezurrten Sicherheitsregeln: `release()` committet niemals implizit (vor dem Zurückstellen in den Autocommit-Modus wird eine offene Transaktion zurückgerollt), ein Doppel-Release derselben Verbindung ist idempotent und wird protokolliert, beim Entleihen wird jede gepoolte Verbindung validiert (tote Verbindungen nach einem DB-Neustart werden still verworfen), und das Warten auf eine freie Verbindung ist auf ein Zeitlimit begrenzt.

Für die Produktion ist dieser Pool ausdrücklich nicht gedacht — der Klassenkommentar sagt es selbst. Sein Wert für Lernende: Man kann in 250 Zeilen lesen, welche Fallen ein Pool bereithält.

`UcpConnectionPool`

`de.allgemein.tx.UcpConnectionPool · 155 Zeilen`

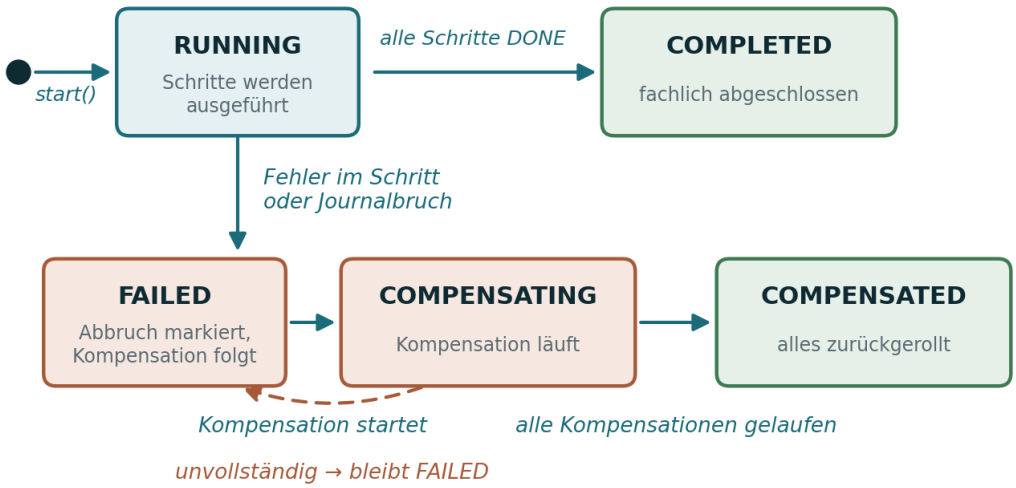
Der Produktions-Pool auf Basis des Oracle Universal Connection Pool (UCP): Validierung beim Entleihen, begrenztes Warten auf freie Verbindungen und JDBC-Connect- sowie Read-Timeouts gegen hängende Aufrufe. Fast Connection Failover und RAC-Lastverteilung sind bewusst nicht konfiguriert — der Pool bedient eine Single-Instance-Standardumgebung; die Erinnerung daran steht im Kommentar, damit niemand eine Fähigkeit vermutet, die nicht da ist.

2.5 Das Saga-Paket tx.saga — lange Geschäftsvorgänge

Eine Saga ist ein langer Geschäftsvorgang, der nicht in einer Datenbanktransaktion laufen kann, ohne Minuten lang Sperren zu halten. Stattdessen wird er in eine Kette kurzer Transaktionen zerlegt: Jeder Schritt committet für sich und wird im Journal festgehalten. Scheitert ein Schritt, werden alle bisherigen Schritte rückwärts kompensiert (Budget zurückbuchen, statt alles zu verwerfen). Das Journal auf den Tabellen `TX_SAGA` und `TX_SAGA_STEP` sichert die Unit of Work über die vielen Einzelcommits hinweg — funktional das, was beim Vorbild das CICS-Journal leistete.

Das Sicherheitsnetz ist das Write-Ahead-Prinzip: Erst wird der Schritt im Journal als `IN_FLIGHT` begonnen (eigener kurzer Commit), dann läuft der Fachschritt, und die Markierung `DONE` wird in derselben Transaktion wie die Fachwirkung committet. Ein Absturz kann daher keinen committeten Schritt mehr verstecken; beim Wiederanlauf gilt `IN_FLIGHT` konservativ als möglicherweise committet und wird mit kompensiert — deshalb müssen alle Kompensationen idempotent sein.

Die drei folgenden Abbildungen zeigen, was das konkret bedeutet: erst den Lebenszyklus der ganzen Saga, dann die Zustände eines einzelnen Schritts samt Zeitachse, zuletzt den Ablauf des Wiederanlaufs.



Der Wiederanlauf behandelt RUNNING, FAILED und COMPENSATING gleichermaßen: fachlich fertig → COMPLETED · nichts zu kompensieren → FAILED · sonst Kompensation. Bleibt nur die COMPLETED-Markierung selbst aus, ist die Saga fachlich fertig: Sie bleibt RUNNING — der nächste Wiederanlauf schließt sie ab (keine Fehl-Kompensation einer erfolgreichen Saga).

Abbildung 1: Lebenszyklus einer Saga im Journal (TX_SAGA.STATE)

So liest man das Bild: Oben der Erfolgsweg — die Saga startet als `RUNNING` und wird `COMPLETED`, sobald alle Schritte DONE sind. Unten der Fehlerpfad: Scheitert ein Schritt (oder bricht die Journalverbindung weg), wird die Saga als `FAILED` markiert und startet sofort die Kompensation (`COMPENSATING`). Die FAILED-Markierung läuft bewusst VOR der Kompensation und darf sie niemals verhindern — ihr eigenes Scheitern wird nur protokolliert. Am Ende steht `COMPENSATED` (alles zurückgerollt) oder erneut `FAILED` (unvollständig; der nächste Wiederanlauf setzt tiefer an, siehe Abbildung 3).

Der Fußnotentext unten nennt den feinsten Fall: Ist die Saga fachlich fertig, aber ausgerechnet die COMPLETED-Markierung schlug fehl, bleibt sie `RUNNING` — der Wiederanlauf erkennt an den Schritt-Zuständen, dass nichts zu kompensieren ist, und schließt sie nur noch ab. So kann kein Journalfehler eine erfolgreiche Saga fälschlich zurückrollen.

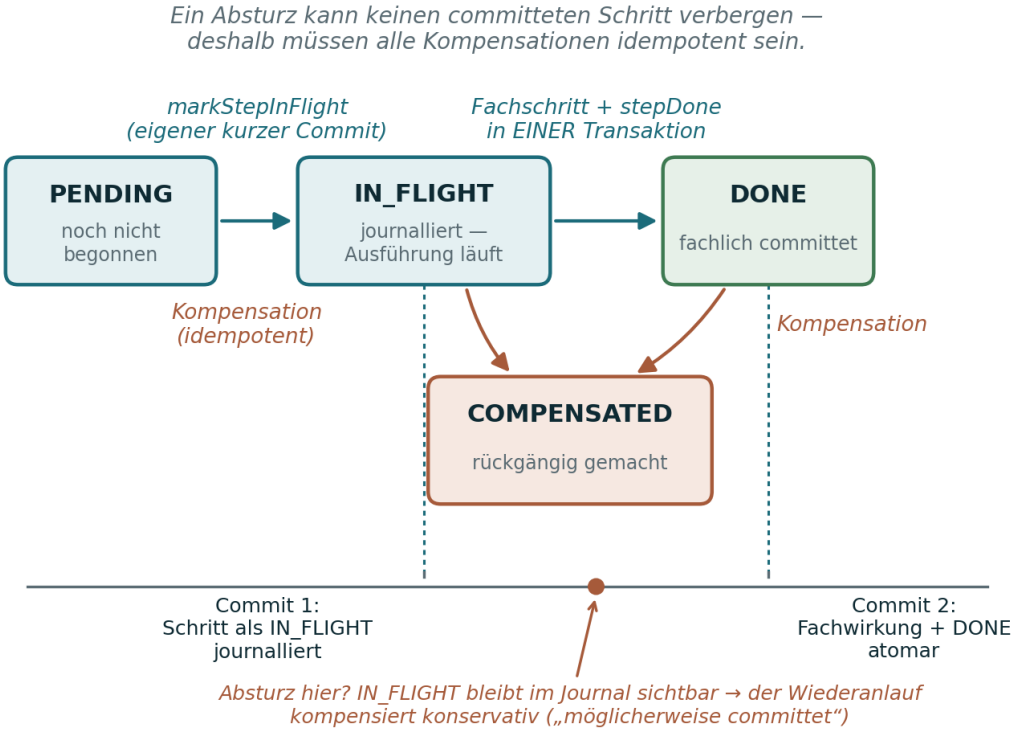
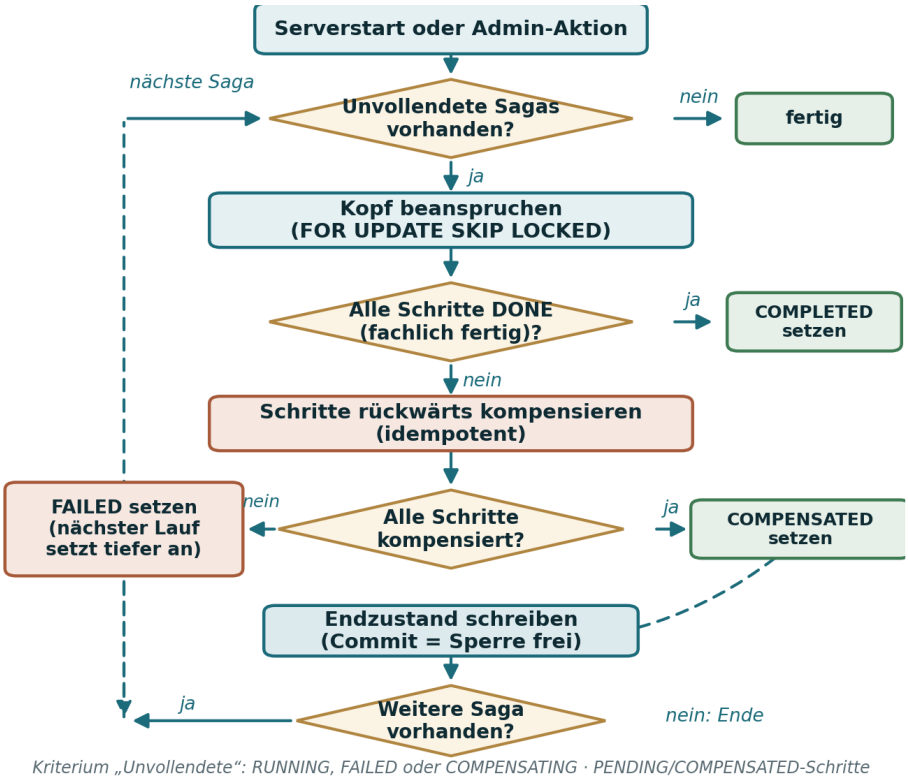


Abbildung 2: Zustände eines Saga-Schritts mit Write-Ahead-Zeitachse (TX_SAGA_STEP.STATE)

So liest man das Bild: Ein Schritt beginnt als `PENDING`. Vor der eigentlichen Arbeit wird er als `IN_FLIGHT` journalliert — das ist **Commit 1**, eine eigene winzige Transaktion. Dann läuft der Fachschritt, und die `DONE`-Markierung wird IN derselben Transaktion geschrieben (**Commit 2**) — Journalstand und Fachwirkung können nicht auseinanderlaufen. Die gestrichelten Linien in der Zeitachse unten verbinden die beiden Commits mit ihren Zuständen.

Der rote Punkt markiert die gefährliche Stelle: Bricht der Server genau zwischen Commit 1 und Commit 2 ab, ist der Schritt `IN_FLIGHT` — und niemand weiß, ob die Fachwirkung durch ist. Das Journal klärt das nicht, sondern entscheidet **konservativ**: IN_FLIGHT gilt als „möglicherweise committet“ und wird kompensiert. Weil die Kompensation damit auch einen nicht-committeten Schritt trifft, muss sie idempotent sein — wiederholtes Ausführen ist harmlos (die roten Pfeile zu `COMPENSATED` gelten von IN_FLIGHT und DONE gleichermaßen).



Kriterium „Unvollendete“: RUNNING, FAILED oder COMPENSATING · PENDING/COMPENSATED-Schritte werden übersprungen, IN_FLIGHT gilt als „möglicherweise committet“ und wird mitkompensiert.

Abbildung 3: Flussdiagramm des Wiederanlaufs (recover) über alle halbfertigen Sagas

So liest man das Bild: Beim Serverstart (oder per Knopf im Adminbereich) werden alle Sagas mit Zustand RUNNING, FAILED oder COMPENSATING nacheinander abgearbeitet. Der Kopf jeder Saga wird mit `SELECT … FOR UPDATE SKIP LOCKED` beansprucht — arbeitet ein zweiter Wiederanlauf parallel, bekommt er diese Saga einfach nicht und nimmt die nächste; niemand blockiert niemanden.

Je Saga gilt: Ist jeder Schritt DONE, war nur die Abschlussmarkierung ausgeblieben — die Saga wird COMPLETED und fertig. Sonst läuft die Kompensation rückwärts über die Schritte, wobei `PENDING` (lief nie) und bereits `COMPENSATED` übersprungen und `IN_FLIGHT` konservativ mitkompensiert werden. Klappt alles, wird COMPENSATED gesetzt, sonst FAILED — der nächste Lauf setzt an der übrigen Stelle wieder an. Der Endzustand wird mit dem Commit geschrieben; genau dadurch wird die Sperre freigegeben.

SagaEngine

`de.allgemein.tx.saga.SagaEngine · final class · 368 Zeilen`

Der Motor: führt eine `SagaDefinition` Schritt für Schritt aus — je Schritt Write-Ahead-Markierung, Fachtransaktion, DONE-Vermerk —, kompensiert bei Fehler alle fertigen Schritte rückwärts und bietet `recover()` an, um halbfertige Sagas nach einem Absturz sauber zu beenden. Der Serverstart der Fachschicht ruft genau diesen Wiederanlauf über alle Saga-Typen auf.

Die Engine selbst kennt keine Fachbegriffe: Sie ordnet Schritte an, achtet auf das Journal und behandelt Fehler. Alles Fachliche steckt in den Aktionen, die die Fachschicht als `SagaDefinition`

übergibt — deshalb kann die Demo (Kapitel 2.8) und die Fachschicht (Kapitel 3.7) dieselbe Engine nutzen.

SagaDefinition

`de.allgemein.tx.saga.SagaDefinition · final class (Builder)`

Die feste Abfolge der Schritte eines Geschäftsvorgangs, gebaut über einen Builder: Typname (zum Wiederfinden im Journal), Liste der Schritte, Payload als Zeichenkette. Ein Beispiel: Vorgang erfassen, Bedarfsprüfung, Bescheid erteilen, Erstauszahlung. Die Definition ist unveränderlich — eine laufende Saga ändert ihr Programm nicht mehr.

SagaStep

`de.allgemein.tx.saga.SagaStep · record (name, action, compensation)`

Ein Schritt der Saga: Name, Aktion und — das Entscheidende — die Kompensation, die ihn rückgängig macht. Jede Kompensation muss idempotent sein, denn der Wiederanlauf führt sie erneut aus, wenn der Schritt möglicherweise schon committet war.

SagaAction

`de.allgemein.tx.saga.SagaAction · @FunctionalInterface`

Funktionale Schnittstelle für Aktion und Kompensation: `run(Connection, payload, sagaId)`. Neben dem Payload erhält die Aktion die `sagaId` — darüber kann sie fachliche Korrelationen deterministisch ableiten (die Saga-Zahlung trägt etwa die Vorgangsnummer SAGA- plus Kennung), sodass eine Kompensation später gezielt nur die eigenen Daten dieser Saga erfasst und fremde unberührt lässt.

TxSagaJournal

`de.allgemein.tx.saga.TxSagaJournal · interface`

Das Journal-Interface der Engine: Saga anlegen, Schritt als IN_FLIGHT markieren, DONE in der Fachtransaktion vermerken, unvollendete Sagas auflisten und exklusiv beanspruchen. Die Engine hängt nur an dieser Abstraktion — im Test (`SagaEngineTest`) wird das Journal durch eine In-Memory-Umsetzung ersetzt, ohne dass die Engine es merkt.

SagaJournal

`de.allgemein.tx.saga.SagaJournal · 474 Zeilen`

Die Oracle-Umsetzung des Journals auf `TX_SAGA` (Kopf) und `TX_SAGA_STEP` (Schritte). Kopf und Schritte werden atomar angelegt (RUNNING/PENDING); alle Zustandsübergänge tragen Guards, sodass ein verlorenes Wettrennen nichts kaputt macht, sondern wirft oder harmlos ist. Das Journal-Dedup gegen doppelte Saga-IDs läuft transaktional auf der Datenbank, nicht im Speicher.

SagaClaim

`de.allgemein.tx.saga.SagaClaim · interface · AutoCloseable`

Der exklusive Anspruch auf eine unvollendete Saga während des Wiederlaufs: Die Kopfzeile wird mit `SELECT FOR UPDATE SKIP LOCKED` gesperrt — ein parallel laufender Wiederanlauf überspringt diese Saga und nimmt die nächste. Der Anspruch endet genau einmal: `finish` schreibt den Endzustand und committet, `close` verwirft und rollt zurück; beide geben die Verbindung an den Pool zurück.

UnfinishedSaga

`de.allgemein.tx.saga.UnfinishedSaga · final class`

Wertobjekt: eine halbfertige Saga aus dem Journal, inklusive der Zustände aller Schritte (PENDING, IN_FLIGHT, DONE, COMPENSATED). Es ist die Grundlage des Wiederanlaufs — die Engine entscheidet anhand dieser Zustände, ob vorwärts oder rückwärts weitergearbeitet wird.

SagaState

`de.allgemein.tx.saga.SagaState · enum`

Der Lebenszyklus der ganzen Saga: RUNNING, COMPLETED, FAILED, COMPENSATING, COMPENSATED. Der Unterschied zwischen FAILED und COMPENSATED ist wichtig: FAILED heißt abgebrochen (der Wiederanlauf läuft noch oder hatte nichts zu tun), COMPENSATED heißt vollständig zurückgerollt.

2.6 Das Audit-Paket tx.audit

Revisionssicherheit quer zur Ausführung: Commit, Rollback, Retry, Timeout, Saga-Schritte und Kompensationen werden als Ereignisse protokolliert. Der Datenschutz-Grundsatz lautet: keine Nutzdaten — Payloads mit Sozialdaten erscheinen im Audit-Log nie; die Korrelation läuft allein über Saga-ID und Transaktions-ID.

TxAuditEvent

`de.allgemein.tx.audit.TxAuditEvent · record`

Ein einzelnes revisionssicheres Ereignis: Art, Zeitpunkt, IDs und Zusatzangaben (etwa die Fehlerklasse). Als Record bewusst schlank und unveränderlich — Audit-Einträge nachträglich zu ändern wäre ein Widerspruch in sich.

TxAuditListener

`de.allgemein.tx.audit.TxAuditListener · interface`

Die Schnittstelle für Empfänger von Audit-Ereignissen: Datei-Logger, SIEM-Anbindung oder der Ringpuffer. Über dieses Interface kann ein Betrieb später eigene Senken anschließen, ohne die Steuerung anzufassen.

TxAuditLog

`de.allgemein.tx.audit.TxAuditLog · 271 Zeilen`

Das Audit-Log selbst: Ereignisse gehen an einen strukturierten Logger, in einen thread-sicheren In-Memory-Ringpuffer der letzten 1.000 Einträge (für Oberfläche und Diagnose) und optional in eine Datei auf Platte, die bei etwa 10 MB rotiert (ein Vorgänger wird gehalten). Der Writer bleibt dauerhaft geöffnet — jede Zeile einzeln öffnen und schließen wäre auf Dauer zu teuer.

2.7 Das Monitoring-Paket tx.monitoring (JMX)

JMX ist die Standard-Überwachungsschnittstelle des JDK: Komponenten melden Kennzahlen als MBeans, und Werkzeuge wie JConsole oder VisualVM zeigen sie live an. Das Paket besteht aus Paaren — je ein MBean-Interface (was ist abfragbar?) und ein Adapter (wie kommt man an die Werte?).

TxMonitoring

`de.allgemein.tx.monitoring.TxMonitoring · final class`

Die zentrale Registrierung: meldet TaskController, UCP-Pool und generische Pools als MBeans beim JDK-MBeanServer an. Ein Aufruf beim Serverstart genügt, danach sind Parallelität, Durchlauf, Fehler, Timeouts und Pool-Auslastung über JConsole sichtbar.

TaskControllerMBean

`de.allgemein.tx.monitoring.TaskControllerMBean · interface`

Die JMX-Schnittstelle für den TaskController (Vorbild: CICS-Transaktionsüberwachung): Statistik und Laufzeitzahlen zum Abfragen über beliebige Monitoring-Werkzeuge.

TaskControllerAdapter

`de.allgemein.tx.monitoring.TaskControllerAdapter · final class`

Der Standard-Adapter, der das MBean-Interface für den TaskController implementiert, indem es die `TaskStats` durchreicht. Er übersetzt zwischen der JMX-Welt (Attribute, einfache Typen) und der Klasse selbst.

UcpConnectionPoolMBean

`de.allgemein.tx.monitoring.UcpConnectionPoolMBean · interface`

Die JMX-Schnittstelle für den Oracle-UCP-Pool: ausgeliehene und freie Verbindungen sowie die Grenzen — die Fragen, die man im Betrieb zuerst stellt, wenn die Anwendung langsam wird.

UcpConnectionPoolAdapter

`de.allgemein.tx.monitoring.UcpConnectionPoolAdapter · final class`

Der Adapter für den UCP-Pool; er liest die Poolkennzahlen und stellt sie als MBean-Attribute bereit.

GenericConnectionPoolMBean

`de.allgemein.tx.monitoring.GenericConnectionPoolMBean · interface`

Die generische JMX-Schnittstelle für beliebige Verbindungspools (etwa den SimpleConnectionPool in der Entwicklung): die üblichen Kennzahlen ohne herstellerspezifische Typen.

GenericConnectionPoolAdapter

`de.allgemein.tx.monitoring.GenericConnectionPoolAdapter · final class`

Der zugehörige Adapter; er ummantelt jeden Pool, der die Grundkennzahlen hergibt, und meldet ihn unter JMX.

2.8 Das Demo-Paket tx.demo

Die Demo zeigt Kern und Engine im Zusammenspiel auf zwei kleinen Tabellen (`TX_DEMO_ANTRAG`, `TX_DEMO_ZAHLUNG`) — ohne Fachschicht. Sie ist der schnellste Weg, eine Saga einmal leben zu sehen: `start.bat demo` oder `mvn compile exec:java`.

DemoMain

`de.allgemein.tx.demo.DemoMain · final class`

Der Einstiegspunkt der Demo: baut Pool, TxManager, TaskController und SagaEngine auf, konfiguriert über die Umgebungsvariablen TX_URL, TX_USER und TX_PASS, und führt dann die Antrag-Saga plus parallele Tasks über die Task-Steuerung vor. Wer den Aufbau einer lauffähigen Instanz sehen will (Reihenfolge Pool, Manager, Engine, Wiederanlauf), liest zuerst diese Klasse.

AntragsDemoProzess

`de.allgemein.tx.demo.AntragsDemoProzess · final class`

Die Saga-Definition der Demo (Antrag, Bescheid, Auszahlung) samt Schritten und Kompensationen auf den Demo-Tabellen. Ausgelagert aus der DemoMain, damit auch der Adminbereich der Fachschicht-Anwendung dieselbe Definition für den Wiederanlauf-Aktion nutzen kann. Alle Schritte und Kompensationen korrelieren über die mitgespeicherte SAGA_ID, nicht über den Namen — parallele Demos mit gleichem Namen können sich dadurch gegenseitig nichts wegkompensieren.

3 Die Fachschicht SDN.Leistung: de.allgemein.lmg

3.1 Idee und Zusammenspiel

Die Fachschicht setzt die Transaktionssteuerung für die Grundsicherung im Alter und bei Erwerbsminderung (SGB XII) ein. Die Schichtung ist streng: Die **Webschicht** (`web`) nimmt Anfragen entgegen und prüft Sitzung und Rolle; die **Fachdienste** (`service`) ordnen die fachliche Arbeit als kurze Transaktionen über den TxManager an; die **Datenzugriffe** (`dao`) sprechen die Oracle-Tabellen auf der Verbindung der laufenden Transaktion; die **Berechnung** (`calc`) ist reines Fachrecht ohne Datenbank; die **Wertobjekte** (`model`) tragen die Daten.

Die fachlichen Tabellen: `SOZIAL_FALL` (Leistungsberechtigte), `BEDARFSGEMEINSCHAFT` (Vorgänge mit Budget und Bankverbindung), `ZAHLUNG` (Auszahlungen mit Vier-Augen-Workflow), `BESCHEID` (Bewilligungsbescheide mit JSON-Snapshot), `JOURNAL` (revisionssicheres Wer-wann-was) und `SATZSTAND` (Regelbedarfsstufen je Jahr).

3.2 model — die Wertobjekte der Fachwelt

Alle Modellklassen sind unveränderliche Records: Daten werden gelesen, nie an Ort und Stelle verändert — Änderungen laufen immer über die DAOs als neue schreibende Transaktion. Das macht Objekte im Umlauf unempfindlich gegen Nebenläufigkeit.

SozialFall

`de.allgemein.lmg.model.SozialFall · record · 141 Zeilen`

Ein Leistungsberechtigter innerhalb einer Bedarfsgemeinschaft — der zentrale Fachbegriff des Projekts. Der Record trägt alle Parameter der Bedarfs- und Einkommensermittlung (Paragrafen 27a, 30, 82 SGB XII): Regelbedarfsstufe, Merkzeichen G, Schwangerschaft ab der 13. Woche, Alleinerziehend-Merkmal, dezentrale Warmwassererzeugung, Rente, Erwerbseinkommen und die Absetzbeträge.

Die VERSION-Spalte schützt die Maskenbearbeitung gegen verlorene Aktualisierungen zweier Sachbearbeiter (optimistisches Locking). Besonders ist das Feld `entbindungsmonat`: Es beendet den Schwangeren-Mehrbedarf fachlich zum Ende des Entbindungsmonats — nur dieser Mehrbedarf entfällt danach, die übrige Leistung bleibt berechenbar.

Bedarfsgemeinschaft

`de.allgemein.lmg.model.Bedarfsgemeinschaft · record`

Der geführte Vorgang mit eindeutigem Aktenzeichen. Er trägt das verfügbare Restbudget pro Monat (bei jeder angeordneten Auszahlung gemindert, bei Storno zurückgebucht), die Standard-Bankverbindung (IBAN, BIC, Kontoinhaber) und das Gesamtvermögen für die Vermögensanrechnung nach Paragraf 90 SGB XII — das Schonvermögen (10.000 EUR je volljähriger Person zuzüglich 500 EUR je unterhaltener Person) wird in der Berechnung abgezogen.

Auch hier schützt eine VERSION-Spalte gegen Gleichzeitigkeitskonflikte, und eine Historisierungskennzeile hält alte Stände nachvollziehbar, statt sie zu löschen.

Zahlung

`de.allgemein.lmg.model.Zahlung · record`

Eine angeordnete Auszahlung. Der Status durchläuft ANGELEGT, BEREIT, UEBERGEBEN oder endet STORNIERT — die Zahlung entsteht immer im Status ANGELEGT und wartet auf die Freigabe durch eine zweite Person (Vier-Augen-Prinzip, serverseitig erzwungen). Die VORGANGS_NR ist eine UUID aus dem Frontend und macht den Buchungsvorgang idempotent: derselbe Request kann nie doppelt buchen.

Das Feld `vermoegenseinsatz` dokumentiert den bei der Anordnung verbrauchten verwertbaren Vermögensanteil (Verbrauchsfortschreibung nach Paragraf 90 SGB XII) — das Storno bucht genau diesen dokumentierten Betrag zurück, neben dem Restbudget auch ins Vermögen.

Bescheid

`de.allgemein.lmg.model.Bescheid · record`

Der Bewilligungsbescheid nach Paragraf 44 SGB XII: Bewilligungszeitraum, Zahlbetrag des ersten Monats, Status ERTEILT oder AUFGEHOBEN. Im Feld BERECHNUNG liegt der versionierte JSON-Snapshot des Berechnungsergebnisses aus dem Erteilungszeitpunkt — der Nachdruck zeigt ausschließlich diesen Stand, auch wenn die SozialFälle später geändert werden (revisionssicherer Nachdruck). Die Snapshot-Version 2 enthält zusätzlich die verbindlichen Einzelbeträge aller Bewilligungsmonate.

Journaleintrag

`de.allgemein.lmg.model.Journaleintrag · record`

Ein Eintrag im revisionssicheren Buchungsjournal: Wer hat wann was angeordnet? Die Referenz auf die Bedarfsgemeinschaft ist bewusst nur logisch (kein Fremdschlüssel), damit Einträge auch nach der Historisierung des Vorgangs erhalten bleiben — Nachvollziehbarkeit überdauert den Lebenszyklus des Vorgangs.

VorgangslisteEintrag

`de.allgemein.lmg.model.VorgangslisteEintrag · record`

Eine Zeile der Fallübersicht: Bedarfsgemeinschaft plus Vorschau des Zahlbetrags, berechnet über alle zugehörigen SozialFälle. Ist die Berechnung nicht möglich (etwa fehlende Regelbedarfsstufe), steht die Vorschau auf null und die Maske zeigt einen Hinweis — die Liste bleibt lesbar.

3.3 dao — der Datenzugriff

Die DAOs folgen einer strikten Regel: Alle Methoden laufen auf der Verbindung der laufenden Transaktion (aus dem TxContext); DAOs öffnen und schließen bewusst keine eigenen Verbindungen, sonst brächen die Transaktionsgrenzen. So entsteht nie ein Mischzustand aus transaktionalen und freien Zugriffen.

Tabelle 2: DAOs und ihre Tabellen

DAO	Tabelle	Besonderheit
SozialFallDao	SOZIAL_FALL	aktualisiere mit VERSION-Prüfung (optimistisches Locking)
BedarfsgemeinschaftDao	BEDARFSGEMEINSCHAFT	BG-Sperre (FOR UPDATE), Budget, Historisierung
ZahlungDao	ZAHLUNG	VORGANGS_NR eindeutig (Idempotenz-Anker)
BescheidDao	BESCHEID	Snapshot-CLOB versioniert speichern und lesen
JournalDao	JOURNAL	revisionssicher, atomar in derselben Transaktion
SatzstandDao	SATZSTAND	größter GUELTIG_AB kleiner gleich Bezugsmonat

`SozialFallDao`

`de.allgemein.lmg.dao.SozialFallDao · 174 Zeilen`

Datenzugriff für SOZIAL_FALL, die Parameter der Bedarfs- und Einkommensermittlung. Maskenänderungen laufen über `aktualisiere` mit VERSION-Prüfung: Zwei Sachbearbeiter, die dieselbe Maske speichern, bekommen keinen stillen Überschreibkonflikt — der zweite erhält einen fachlichen Fehler. Der Buchungsweg liest dagegen im Bann der BG-Sperre (siehe BedarfsgemeinschaftDao), also mit konsistentem Stand.

`BedarfsgemeinschaftDao`

`de.allgemein.lmg.dao.BedarfsgemeinschaftDao · 273 Zeilen`

Der größte DAO, weil hier der kritische Buchungspfad liegt: die pessimistische Zeilensperre auf die Bedarfsgemeinschaft (SELECT … FOR UPDATE), die Minderung und Rückbuchung des Restbudgets, die Vermögensverbrauchsfortschreibung sowie Anlage, Änderung und Historisierung des Vorgangs. Fallanlage und Falländerung sind gegen konkurrierende Historisierung abgesichert — zwei gleichzeitige Änderungen können keinen Mischzustand erzeugen.

ZahlungDao

`de.allgemein.lmg.dao.ZahlungDao · 252 Zeilen`

Datenzugriff für ZAHLUNG. Die VORGANGS_NR ist eindeutig — sie ist der Idempotenz-Anker gegen Doppelbuchungen bei wiederholten Requests (Doppelklick, Netzwerk-Wiederholung). Statuswechsel sind bewusst guarded: Ein Übergang, der im Workflow nicht vorgesehen ist (etwa UEBERGEBEN direkt freigeben), wirft statt still zu passieren. Der Kassenlauf holt sich hier die gesperrte Menge der freigegebenen Zahlungen.

BescheidDao

`de.allgemein.lmg.dao.BescheidDao · 131 Zeilen`

Datenzugriff für BESCHEID, einschließlich des versionierten JSON-Snapshots im CLOB-Feld BE-RECHNUNG. Gespeichert wird beim Erteilen, gelesen beim Nachdruck — ältere Snapshot-Versionen (v1) bleiben lesbar, auch wenn der Schreibweg bereits v2 erzeugt.

JournalDao

`de.allgemein.lmg.dao.JournalDao · final class`

Datenzugriff für das Buchungsjournal (JOURNAL). Jede fachlich relevante Aktion schreibt hier einen revisionssicheren Eintrag — und zwar in derselben Transaktion wie die Änderung selbst: Journal und Fachwirkung stehen oder fallen zusammen. Neben der Sachbearbeiter-Kennung läuft hier auch das Einsichtsprotokoll mit (60-Sekunden-Bündelung gleicher Kennungen, damit Masken-Reloads das Journal nicht fluten).

SatzstandDao

`de.allgemein.lmg.dao.SatzstandDao · final class`

Datenzugriff auf SATZSTAND, die je Jahr gepflegten Regelbedarfsstufen (Anlage zu Paragraf 28 SGB XII). Für einen Bezugsmonat gilt der zuletzt wirksam gewordene Stand: der größte GUELTIG_AB kleiner gleich Monat. Neue Zeilen gelten ab ihrem Gültigkeitsmonat; alte bleiben für zurückliegende Monate lesbar — der Jahreswechsel ist ein INSERT, keine Codeänderung. Ohne passenden Stand weist der Service fachlich ab, statt mit erfundenen Sätzen zu rechnen.

3.4 calc — Bedarfsberechnung nach SGB XII

Das Berechnungspaket ist das fachliche Herz: hier steht das Recht in Code. Die Berechnung ist deterministisch und ohne Datenbank — sie erhält die Parameter und liefert das Ergebnis. Genau deshalb lässt sich jede Regel mit handgerechneten Werten testen (`GoldWerteTest`).

Leistungsberechnung

`de.allgemein.lmg.calc.Leistungsberechnung · 387 Zeilen`

Die komplette Bedarfsberechnung der Grundsicherung: Regelbedarf nach der Regelbedarfsstufe; Mehrbedarfe nach Paragraf 30 SGB XII (Merkzeichen G mit 17 Prozent, Schwangerschaft ab der 13.

Woche mit 17 Prozent, Alleinerziehende mit 36 bzw. 12 Prozent und Deckel, Obergrenze: Summe höchstens der Regelbedarf; dezentrales Warmwasser stufenabhängig zwischen 0,8 und 2,3 Prozent); Kosten der Unterkunft als Kaltmiete plus Betriebskosten plus Heizkosten mit Angemessenheitsgrenze je Bedarfsgemeinschaft; Einkommensanrechnung nach Paragraf 82 mit Absetzbeträgen und Erwerbsfreibetrag (höchstens 50 Prozent der Stufe 1); Vermögensanrechnung nach Paragraf 90 mit Schonvermögen.

Die Klassendokumentation listet alle Sätze als Formeln — das ist die beste Nachschlagestelle, wenn die Frage lautet: Wie setzt sich dieser Betrag zusammen?

Satzstand

`de.allgemein.lmg.calc.Satzstand · record (gueltigAb, Beträge je Stufe)`

Die Regelbedarfsstufen eines Gültigkeitszeitraums. Weil die Sätze jährlich zum 1. Januar per Verordnung fortgeschrieben werden, liegen sie in der Datenbanktabelle SATZSTAND — eine Zeile je Gültigkeitsmonat —, nicht als Konstanten im Code. Die Konstante `REFERENZ_2026` ist der Seed-Stand (und dient DB-unabhängigen Kompatibilitätsüberladungen sowie dem Simulator); der Produktionspfad liest den Stand stets aus der Datenbank zum Bezugsmonat.

BerechnungsSnapshot

`de.allgemein.lmg.calc.BerechnungsSnapshot · 221 Zeilen`

Serialisiert das Berechnungsergebnis einer Bescheiderteilung als versioniertes JSON und liest es verlustfrei zurück. Der Snapshot wird mit dem Bescheid gespeichert, damit der Nachdruck unabhängig von späteren Falländerungen stets den Zustand des Erteilungszeitpunkts zeigt. Alle Beträge werden als Zeichenketten fester Skala geführt, damit die BigDecimal-Skalierung den Speicher-Roundtrip exakt überlebt — kein Cent darf beim Serialisieren verrutschen.

Bewilligungsplan

`de.allgemein.lmg.calc.Bewilligungsplan · final class`

Die monatsbezogene Bewilligung: Für jeden Monat des Bewilligungszeitraums steht der verbindliche Einzelbetrag, einschließlich bekannter Satzwechsel und Bedarfsänderungen (etwa das Ende des Schwangeren-Mehrbedarfs ab dem Folgemonat der Entbindung). Der Plan ist der Inhalt von Snapshot-Version 2 und die Grundlage, mit der Bescheid und Auszahlungen auch über Monatsgrenzen hinweg zusammenpassen.

3.5 config — externalisierte Konfiguration

LmgConfig

`de.allgemein.lmg.config.LmgConfig · 193 Zeilen`

Die externalisierte Konfiguration des Leistungsmanagements, geladen als Prioritätskaskade (12-Factor-Prinzip): System-Properties schlagen Umgebungsvariablen, diese schlagen die Datei lmg.properties, und am Ende stehen Standardwerte. Über die Kaskade konfiguriert sind unter

anderem Datenbankverbindung, Port und Bind-Adresse (Standard Loopback), die Benutzerliste, Sitzungs-Timeout und Anmelde-Drosselung, die örtlichen KdU-Parameter (Warmwasser-Abzug, Angemessenheitsgrenze) und der Text der Rechtsbehelfsbelehrung. Die Datei wird UTF-8-gelesen — Umlaute sind kein Glücksspiel.

3.6 service — die Fachdienste

Die Fachdienste sind die Schnittstelle zwischen Webschicht und Transaktionssteuerung. Das Interface `LeistungsService` beschreibt den Vertrag, `SozialLeistungsService` implementiert ihn; die übrigen Klassen sind Hilfstypen (Aufträge, Ergebnisse, Validierung, Bescheiddruck). Jede Dienst-Methode ist eine abgeschlossene Arbeitseinheit — eine kurze Oracle-Transaktion.

LeistungsService

`de.allgemein.lmg.service.LeistungsService · interface · 117 Zeilen`

Das Fach-Interface: Fall anlegen und ändern, Vorgangslisten und -details lesen, Auszahlung anordnen, freigeben, stornieren, Kassenlauf starten, Bescheid erteilen und drucken, Systemstatus. Die Webschicht kennt nur dieses Interface — dadurch bleibt sie testbar und die Implementierung austauschbar. Jede Methode entspricht einer abgeschlossenen Arbeitseinheit.

SozialLeistungsService

`de.allgemein.lmg.service.SozialLeistungsService · 894 Zeilen`

Die mit Abstand größte Klasse: die Fachschicht der Leistungsverarbeitung. Alle Aktionen laufen als kurze Oracle-Transaktionen über den TxManager. Der kritische Buchungsweg `ordneAuszahlungAn` zeigt das ganze Muster auf einmal: Idempotenz über die Vorgangsnummer, pessimistische Zeilensperre auf die Bedarfsgemeinschaft, Budget-Prüfung, Anlage der Zahlung im Status ANGELEGT (Vier-Augen: die Freigabe erfolgt immer durch eine zweite Person), Vermögensverbrauchsfortschreibung, Minderung des Restbudgets und revisionssicherer Journaleintrag — alles unteilbar in einer Transaktion.

Dazu kommen: Fallanlage und -änderung mit Historisierung, die Einsichtsprotokollierung vollständiger Falldaten (60-Sekunden-Bündelung), die Bescheiderteilung mit Snapshot und Monatsplan, Storno mit Zahlungssperre und Restbudget-Rückbuchung, der Kassenlauf (übergibt exakt die gesperrte Menge) und die Abstimmung von Bescheid und Zahlung bei bekannten Satzwechseln.

Der Statuswechsel einer Zahlung ist bewusst klein gehalten — der folgende Zustandsautomat zeigt alle erlaubten Wege und die Absicherungen an den Übergängen:

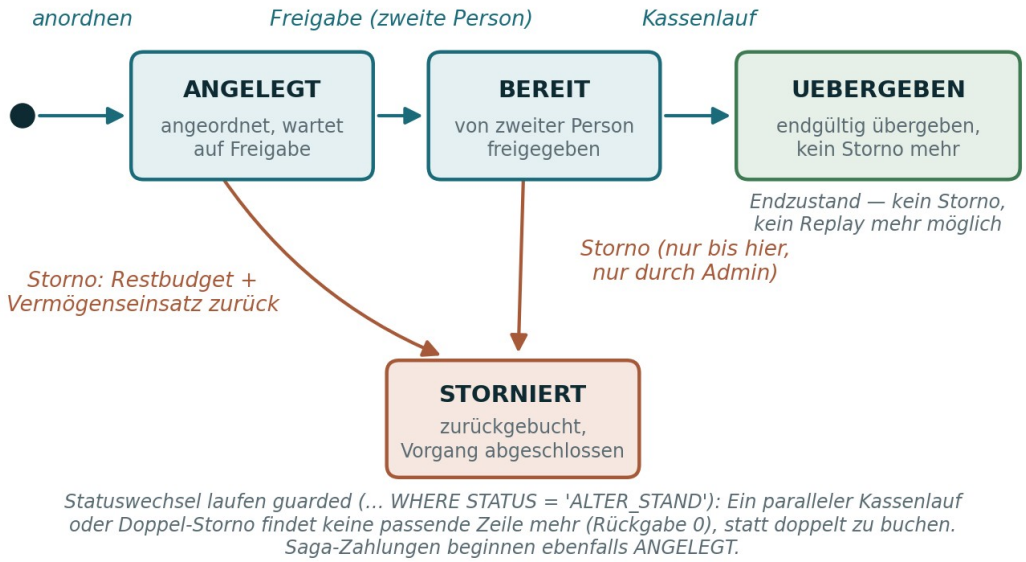


Abbildung 4: Zustandsautomat einer Zahlung (ZAHLUNG.ZAHLUNGSSTATUS)

So liest man das Bild: Jede Zahlung entsteht als `ANGELEGT` — auch die der Saga (Abschnitt 3.7). Die Freigabe auf `BEREIT` ist das Vier-Augen-Prinzip: Sie erfolgt serverseitig und nur durch eine zweite Person, Erfasser und Freigebender müssen verschieden sein; eine Selbstfreigabe wird abgewehrt, nicht nur ausgegraut. Der `Kassenlauf` überführt die gesperrte Menge aller BEREIT-Zahlungen auf `UEBERGEBEN` — dieser Status ist ein Endzustand: Von hier gibt es kein Storno und kein Replay mehr, eine korrigierte Zahlung braucht eine neue Vorgangsnummer.

Zwei Absicherungen sind im Bild unten angemerkt: Alle Statuswechsel laufen **guarded** (UPDATE … WHERE STATUS = 'ALTER_STAND') — ein paralleler Kassenlauf oder ein Doppel-Storno findet keine passende Zeile mehr und wird als Rückgabe 0 erkannt, statt doppelt zu buchen. Und das **Storno** bucht nicht nur das Restbudget zurück, sondern genau den bei der Anordnung dokumentierten Vermögenseinsatz — die Verbrauchsfortschreibung des Vermögens (§ 90 SGB XII) wird also exakt rückgängig gemacht, nicht geschätzt.

`FachlicheException`

`de.allgemein.lmg.service.FachlicheException · erbt TxException`

Der fachliche Fehler: kein Anspruch, Restbudget erschöpft, gleichzeitige Maskenänderung zweier Sachbearbeiter. Er erbt von TxException, damit der TxManager ihn unangetastet durchreicht (Rollback ja, Wiederholung nein) und die Webschicht ihn als Fachfehler erkennt — sonst käme er als technischer Fehler (HTTP 500) an. Das Feld `code` steuert die Anzeige in der Oberfläche.

SepaValidator

`de.allgemein.lmg.service.SepaValidator · final class`

Validierung und Formatierung von SEPA-Bankverbindungen: IBAN nach ISO 13616 mit Modulo-97-Prüfziffernverfahren (ISO 7064), BIC nach ISO 9362. Tippfehler in Bankverbindungen werden so vor dem Speichern abgefangen — nicht erst, wenn die Zahlung bei der Bank platzt.

BescheidGenerator

`de.allgemein.lmg.service.BescheidGenerator · 432 Zeilen`

Erzeugt den Bewilligungsbescheid (Paragraf 44 SGB XII) mit amtlichem Layout: Verfügungssatz, fallbezogene Bedarfsaufschlüsselung mit KdU-Vorbehalt, Zahlungsverbindung, Begründung mit Hauptgründen und vollständiger Rechtsbehelfsbelehrung samt Belehrung nach Paragraf 73 SGG. Der Druck zeigt ausschließlich den versionierten JSON-Snapshot aus dem Erteilungszeitpunkt; fehlt er, wird neu gerechnet und ein deutlicher Hinweiskasten gesetzt. Ein Einmal-Druck-Token verhindert den wiederholten Aufruf desselben Links.

Auszahlungsauftrag

`de.allgemein.lmg.service.Auszahlungsauftrag · record`

Die Anordnung einer Auszahlung durch den Sachbearbeiter (der Auszahlung-anordnen-Button): Ziel-Bedarfsgemeinschaft, Betrag, Zahlungsmonat, Bankverbindung und — die Vorgangsnummer, die das Frontend als UUID erzeugt und die den Vorgang idempotent macht.

Auszahlungsergebnis

`de.allgemein.lmg.service.Auszahlungsergebnis · record`

Das Ergebnis einer Auszahlungsanordnung: die gebuchte (oder bei Wiederholung bereits existierende) Zahlung samt neuem Stand der Bedarfsgemeinschaft. Das Feld `bereitsGebucht` bedeutet: Die Vorgangsnummer war schon vorhanden, es wurde nichts erneut gebucht — die Oberfläche kann das dem Sachbearbeiter ehrlich melden.

Kassenlaeufergebnis

`de.allgemein.lmg.service.Kassenlaeufergebnis · record (anzahl, summe)`

Das Ergebnis eines Kassenlaufs: Anzahl und Summe der von BEREIT auf UEBERGEBEN gesetzten Zahlungen — null und null, wenn nichts bereit war. Bewusst schmal: Ein Kassenlauf ist ein einheitlicher Stapelübergang, keine Auswertung.

Systemstatus

`de.allgemein.lmg.service.Systemstatus · record`

Die Momentaufnahme des Systemzustands für den Adminbereich: Erreichbarkeit der Datenbank samt Kennung und Antwortzeit, Bestandszahlen einschließlich der Saga-Stände (offene Sagas zählen RUNNING und COMPENSATING — Wiederanlauf nötig; abgebrochene zählen FAILED), Task-Statistik und Laufzeitumgebung.

Systemdiagnose

`de.allgemein.lmg.service.Systemdiagnose · 152 Zeilen`

Die aktive Systemdiagnose für den Adminbereich: prüft die Datenbank (Erreichbarkeit, Produktkennung, Antwortzeit, Bestandszahlen), liest die Task-Statistik und stößt bei Bedarf den Wiederanlauf halbfertiger Sagas an. Die Prüfung läuft als kurze Transaktion mit hartem Zeitlimit und genau einem Versuch — ein Datenbank-Ausfall soll schnell und ohne Wiederholungslärm gemeldet werden.

Vorgangsdetail

`de.allgemein.lmg.service.Vorgangsdetail · record`

Der vollständige Lesezuschnitt für die Vorgangsmaske: Kopfdaten, alle SozialFälle samt berechneter Bedarfsermittlung, Zahlungshistorie und Journaleinträge. Die Berechnung kann null sein (etwa wenn die Regelbedarfsstufe nicht konfiguriert ist) — die Maske zeigt dann einen Hinweis statt einer Zahl.

Zahlungsmonat

`de.allgemein.lmg.service.Zahlungsmonat · final class`

Eine kleine Wertklasse für den Abrechnungszeitraum im Format JJJJ-MM (Spalte ZAHLUNG.ZAHLUNGSMONAT). Sie verhindert, dass Monatszeichenketten unverbindlich durch die Schichten geistern — Parsing und Formatierung haben genau einen Ort.

3.7 saga — die fachliche End-to-End-Saga

AntragsSagaProzess

`de.allgemein.lmg.saga.AntragsSagaProzess · 773 Zeilen`

Die vollständige fachliche Saga des Leistungsmanagements — vier unteilbare Schritte mit automatischer Rückwärts-Kompensation: erstens Vorgang erfassen (Anlage von Bedarfsgemeinschaft und SozialFällen), zweitens Bedarfsprüfung (SGB-XII-Berechnung; Abbruch bei fehlendem Anspruch), drittens Bescheid erteilen (Paragraf 44), viertens Erstauszahlung anweisen (Zahlung im Status ANGELEGT — auch hier wartet die Vier-Augen-Freigabe).

Drei Konventionen tragen den Wiederanlauf: Die Schritte zwei bis vier rechnen ausschließlich auf dem Datenbestand, den Schritt eins geschrieben hat — nicht auf dem Payload (manuelle Korrekturen zwischen den Schritten sind damit wirksam, und Bescheid wie Auszahlung nutzen dieselbe Basis wie der manuelle Buchungsweg). Alle Kompensationen sind idempotent. Die Saga-Zahlung wird über die deterministische VORGANGS_NR SAGA- plus Kennung korreliert — fremde Zahlungen derselben Bedarfsgemeinschaft bleiben unberührt. Und der Vorgang wird beim Rückwärtslauf historisiert, nicht gelöscht (Paragraf 67 SGB X: auch eine Ablehnung muss nachvollziehbar bleiben).

3.8 web — Weboberfläche und JSON-API

Die Webschicht setzt auf den im JDK enthaltenen HttpServer — bewusst kein Servlet-Container, keine zusätzliche Abhängigkeit. Jede API-Anfrage läuft als Task über den TaskController: begrenzte Parallelität, Wachhund und Statistik der Transaktionssteuerung gelten damit auch für den Webbetrieb.

LmgServer

`de.allgemein.lmg.web.LmgServer · 929 Zeilen · AutoCloseable`

Die größte Klasse des Projekts: die Webschicht mit JSON-API und Oberfläche im SDN.Leistung-Stil. Aufgabenpaket: Sitzungen und Rollen prüfen (Token nur aus dem Header, Ausnahme Druck-Link mit Einmal-Token), Anfragekörper begrenzen, JSON-Tiefen begrenzen, Alle Serverdaten HTML-escaped ausgeben und eine Content-Security-Policy setzen. Sammel-Übersichten liefern keine Bankdaten mehr — IBAN und BIC erscheinen nur im protokollierten Einzelkontext (Einsichtsprotokoll). Druckantworten setzen no-store.

Zum Lesen lohnt der Einstieg über die ROUTEN-Struktur am Anfang der Klasse: Jede Zeile zeigt, welcher Endpunkt auf welche DienstMethode des LeistungsService zeigt.

LmgServerMain

`de.allgemein.lmg.web.LmgServerMain · 161 Zeilen`

Der Startpunkt: baut Pool, TxManager, TaskController, SagaEngine und Fachdienste auf, stößt den Wiederanlauf halbfertiger Sagas über alle Saga-Typen an und startet den Server. Port über LMG_PORT (Standard 8080), Bind-Adresse über LMG_HOST (Standard 127.0.0.1 — bewusst kein Lauschen an allen Schnittstellen; Demo-Standardkonten werden bei Nicht-Loopback verweigert). Vorher die DDL-Skripte einspielen. Ein geordneter Shutdown-Hook sorgt dafür, dass beim Beenden nichts Halbfertiges zurückbleibt.

Benutzerverwaltung

`de.allgemein.lmg.web.Benutzerverwaltung · 213 Zeilen`

Die Anmeldeverwaltung: Benutzerkonten kommen aus der Konfigurationskaskade (Format kennung:passwort:rolle), ohne Angabe gelten Demo-Konten. Die Anmeldung liefert ein Zufalls-Token, mit dem die Webschicht jede Anfrage prüft; Sitzungen liegen im Speicher und laufen nach einem Idle-Timeout ab. Fehlgeschlagene Anmeldeversuche werden gedrosselt (nach fünf Fehlversuchen 30 Sekunden Sperre je Kennung). Für den echten Betrieb gehört hier ein Verzeichnisdienst hinter HTTPS — bewusst nicht selbst gebaut (beim Vorbild machte das der External Security Manager).

Benutzer

`de.allgemein.lmg.web.Benutzer · record (kennung, rolle)`

Der angemeldete Benutzer mit seiner Rolle (admin oder sachbearbeiter). Die Kennung stammt aus der Anmeldung und ist die einzige Quelle für den Sachbearbeiter-Eintrag im Journal — Client-

Angaben werden ignoriert. Die Rolle entscheidet serverseitig über jede Operation (Admin-Sonderrolle über die Rolle, nicht über die Kennung).

Json

`de.allgemein.lmg.web.Json · 348 Zeilen`

Die minimale JSON-Unterstützung — bewusst ohne Bibliothek. Schreiben: Maps, Listen, Zeichenketten, BigDecimal, Booleans und null. Lesen: ein vollständiger Parser (Objekte als LinkedHashMap, Arrays als ArrayList, Zahlen als BigDecimal). Für Robustheit sorgen Grenzen: eine Tiefenbegrenzung und eine Beschränkung von Exponenten gegen Speicher-DoS. Wer einen handgeschriebenen Parser studieren will, findet hier einen sauberen, kleinen.

4 Der Weg einer Auszahlung durch alle Schichten

Um das Zusammenspiel zu verinnerlichen, folgen wir einem konkreten Vorgang: Sachbearbeiter sb-01 ordnet eine Auszahlung an, sb-02 gibt sie frei, der Kassenlauf übergibt sie. Alle genannten Klassen sind in den Kapiteln 2 und 3 beschrieben. Das Sequenzdiagramm zeigt den ersten Teil — die Anordnung — noch einmal als Überblick: Jede Spalte ist eine Schicht, jeder Pfeil ein Aufruf; die Zahlen an den Pfeilen entsprechen den Schritten der nummerierten Liste unter dem Bild.

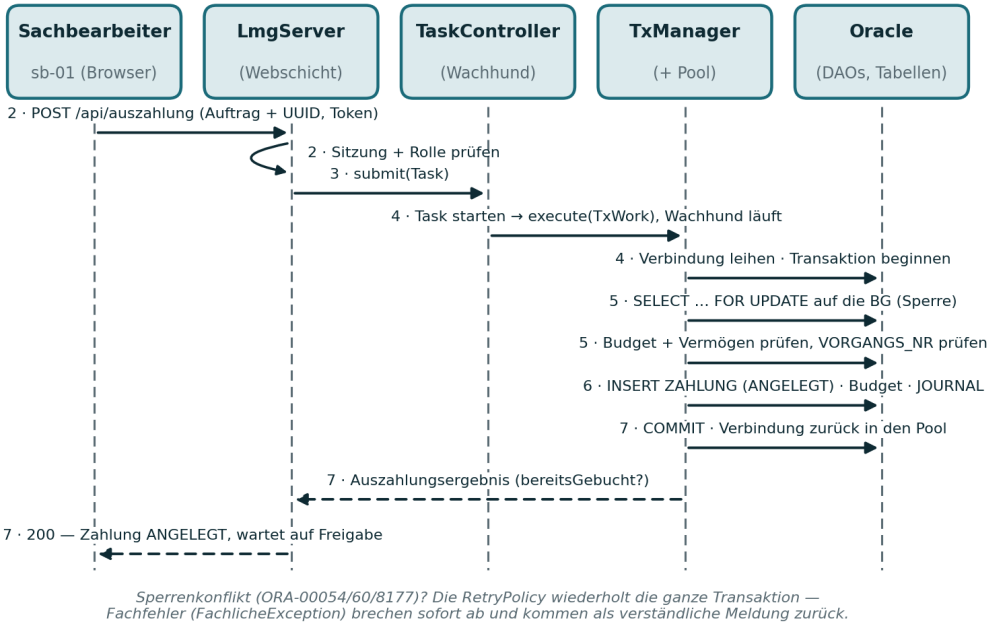


Abbildung 5: Sequenzdiagramm — Auszahlung anordnen, von der Anfrage bis zum Commit

So liest man das Bild: Die Zahlen an den Pfeilen sind die Schritte der Liste unten — tragen mehrere Pfeile dieselbe Zahl, gehören sie zum selben Schritt (bei Schritt 4 etwa das Starten des Tasks und das Beginnen der Transaktion, bei Schritt 5 das Sperren und das Prüfen). Schritt 1 (Anmeldung) liegt vor dem Bild, die Schritte 8 bis 10 (Freigabe, Kassenlauf, Storno/Wiederanlauf) setzen erst nach dem Bild ein. Die beiden gestrichelten Rückpfeile unten sind die Antwortkette; alles davor ist der Hinweg. Wichtig ist die Grenzziehung: Sitzungsprüfung und Task-Start passieren *außerhalb* der Datenbanktransaktion, während die vier Pfeile an Oracle (Sperren, Prüfen, Buchen, Commit) zusammen *eine* Transaktion bilden — genau die, die der `TxManager` führt. Scheitert irgendwas dazwischen mit einem Fachfehler, rollt diese eine Transaktion komplett zurück; ein Sperrenkonflikt löst über die `RetryPolicy` eine ganze Wiederholung ab (die Fußnote im Bild).

1. **Anmeldung:** sb-01 meldet sich an. Die `Benutzerverwaltung` prüft Kennung und Passwort, drosselt Fehlversuche und stellt ein Sitzungs-Token aus. Ab jetzt führt jeder Request dieses Token im Header; der `Benutzer`-Record (Kennung, Rolle) begleitet die Anfrage.

2. **Anfrage trifft ein:** Der `LmgServer` nimmt den JSON-Aufruf entgegen, prüft Sitzung und Rolle, begrenzt Körpergröße und JSON-Tiefe und liest den Payload mit der eigenen `Json`-Klasse — darin ein `Auszahlungsauftrag` mit der vom Frontend erzeugten UUID als Vorgangsnummer.
3. **Task statt Faden:** Die Anfrage wird als Task beim `TaskController` eingereicht. Parallelität ist begrenzt, der Wachhund startet mit der Ausführung — die Wartezeit davor zählt nicht aufs Zeitlimit. Das Ergebnis holt die Webschicht über das `TaskHandle` ab.
4. **Fachdienst und Transaktion:** Der Task ruft `SozialLeistungsService.ordneAuszahlungAn`. Der Service verpackt die Arbeit als `TxWork` und übergibt sie dem `TxManager` samt `TxOptions`. Der Manager leiht eine Verbindung aus dem `ConnectionPool` und beginnt die Transaktion.
5. **Sperren und prüfen:** Auf der Verbindung der laufenden Transaktion (aus dem `TxContext`) sperren die DAOs die Bedarfsgemeinschaft (SELECT ... FOR UPDATE im `BedarfsgemeinschaftDao`), prüfen das Restbudget und die Vorgangsnummer (`ZahlungDao`: eindeutig — ein wiederholter Request läuft ins Leere statt doppelt zu buchen).
6. **Buchen und journalieren:** Die Zahlung entsteht im Status ANGELEGT, das Restbudget mindert sich, der Vermögenseinsatz wird dokumentiert, und der `JournalDao` schreibt den Wer-wann-was-Eintrag — alles in derselben Transaktion. Scheitert etwas mit einem Fachfehler (`FachlicheException`), rollt der TxManager zurück; ein Sperrenkonflikt (ORA-00054/60/8177) würde über die `RetryPolicy` wiederholt.
7. **Commit und Antwort:** Der TxManager committet, die Verbindung geht zurück in den Pool, das `TxAuditLog` vermerkt das Ereignis (ohne Sozialdaten), und die Webschicht bekommt ein `Auszahlungsergebnis` — bei Wiederholung mit bereitsGebucht gleich wahr.
8. **Vier Augen:** sb-02 (oder ein Admin) gibt frei: `freigebenZahlung` prüft, dass Erfasser und Freigebender verschieden sind, und setzt BEREIT. Eine Selbstfreigabe wird abgewehrt — serverseitig, nicht nur im Frontend.
9. **Kassenlauf:** Der Kassenlauf setzt die gesperrte Menge von BEREIT auf UEBERGEBEN und liefert ein `Kassenlaeufergebnis` (Anzahl und Summe). Erst dieser Schritt bedeutet Übergabe an die Zahlstelle.
10. **Falls etwas schiefläuft:** Ein Storno (mit Zahlungssperre und guarded Statuswechsel) bucht Restbudget und dokumentierten Vermögenseinsatz exakt zurück. Und läuft der Server während eines langen Vorgangs in eine Saga (`AntragsSagaProzess`) mitten im Schritt um, findet der Wiederanlauf der `SagaEngine` beim Start den Journalstand (`TX_SAGA`, `TX_SAGA_STEP`) und beendet den Vorgang sauber über `SagaClaim`.

5 Empfohlene Lernreihenfolge

Der Code lässt sich in Etappen erschließen; jede Etappe baut auf der vorherigen auf. Als Begleitung dienen die Klassenkommentare im Code (ausführlich, deutsch) und die Testklassen als Nutzungsbilder. Jeder Schritt nennt am Ende seine Fundstellen — die Seiten der Beschreibung in diesem Dokument und das Verzeichnis im Quellcode, damit niemand blind stochern muss.

1. **Transaktion zuerst:** `TxManager`, `TxWork`, `TxOptions`, `TxException` und `RetryPolicy` lesen — zusammen kaum 400 Zeilen, aber das Fundament. Danach `TxManagerTest` ansehen: Was passiert bei Fehler, was bei Retry? *Fundstellen: Abschnitt 2.2 — S. 5 (TxManager, TxWork), S. 6 (TxOptions, TxException, RetryPolicy) · Code:* `src/main/java/de/allgemein/tx/` *· Test:* `src/test/java/de/allgemein/tx/TxManagerTest.java`
2. **Pools und Tasks:** `ConnectionPool` mit `SimpleConnectionPool` (die Sicherheitsregeln im Kommentar sind eine Lehrstunde), dann `TaskController` mit Wachhund und `TaskStats`. *Fundstellen: Abschnitt 2.3 — S. 7 (TaskController, TaskHandle, TaskStats), Abschnitt 2.4 — S. 8 (ConnectionPool, SimpleConnectionPool) · Code:* `src/main/java/de/allgemein/tx/`
3. **Saga verstehen:** Abschnitt 2.5 dieses Dokuments (S. 8–13), dann im Code `SagaDefinition`, `SagaStep`, `SagaAction`, dann `SagaEngine` und zum Schluss `SagaJournal`. Die Tests `SagaEngineTest` und `SagaParallelRecoverTest` zeigen das Wiederanlaufversprechen in Aktion. *Fundstellen: Abschnitt 2.5 — S. 11 (SagaEngine), S. 12 (SagaDefinition, SagaStep, SagaAction, SagaJournal), S. 13 (SagaClaim) · Code:* `src/main/java/de/allgemein/tx/saga/` *· Tests:* `src/test/java/de/allgemein/tx/saga/`
4. **Demo laufen lassen:** `start.bat demo` ausführen und parallel `DemoMain` und `AntragsDemoProzess` lesen — die kleinste vollständige Saga des Projekts. *Fundstellen: Abschnitt 2.8 — S. 15 (DemoMain, AntragsDemoProzess) · Code:* `src/main/java/de/allgemein/tx/demo/`
5. **Fachbegriffe und Datenmodell:** `SozialFall`, `Bedarfsgemeinschaft`, `Zahlung`, `Bescheid` (Kapitel 3.2) und die Tabellen in `ddl/lmg_schema.sql`. Ohne diese Begriffe bleibt die Fachschicht Kryptogramm. *Fundstellen: Abschnitt 3.2 — S. 16 (SozialFall, Bedarfsgemeinschaft), S. 17 (Zahlung, Bescheid) · Code:* `src/main/java/de/allgemein/lmg/model/` *· Tabellen:* `ddl/lmg_schema.sql`
6. **Berechnung nachvollziehen:** `Leistungsberechnung` Zeile für Zeile mit `doc/rechtsgrundlagen.md` daneben; dann `Satzstand`, `SatzstandDao`, `BerechnungsSnapshot` und `Bewilligungsplan`. Die handgerechneten Erwartungen in `GoldWerteTest` und `doc/regressionstests.md` prüfen das Verständnis. *Fundstellen: Abschnitt 3.4 — S. 19 (Leistungsberechnung, SatzstandDao), S. 20 (Satzstand, BerechnungsSnapshot, Bewilligungsplan) · Code:* `src/main/java/de/allgemein/lmg/calc/` *(SatzstandDao in* `src/main/java/de/allgemein/lmg/dao/`*) · Test:* `src/test/java/de/allgemein/lmg/calc/GoldWerteTest.java`
7. **Dienste und Web:** `LeistungsService` (Interface) und dann `SozialLeistungsService` — am besten direkt mit Kapitel 4 dieses Dokuments als Karte. Abschluss: `LmgServer`,

`Benutzerverwaltung`, `Json`. Zum Vertiefen die Szenario-Regression (`SzenarioRegressionTest`, zehn Fälle) und der Simulator `testsweet`. *Fundstellen: Abschnitt 3.6 — S. 21 (LeistungsService, SozialLeistungsService), Abschnitt 3.8 — S. 25 (LmgServer, Benutzerverwaltung), S. 26 (Json) · Code:* `src/main/java/de/allgemein/lmg/service/` *und* `src/main/java/de/allgemein/lmg/web/` *· Test:* `src/test/java/de/allgemein/lmg/db/SzenarioRegressionTest.java` *· Simulator:* `testsweet/`

Ein Hinweis zum Schluss: Die Architekturregeln aus AGENTS.md (kurze Transaktionen über TxManager, lange über die Saga mit idempotenten Kompensationen, Wiederholung nur bei Sperrenkonflikten, kein XA, bewusst keine Frameworks) erklären viele Gestaltungsentscheidungen, die auf den ersten Blick umständlich wirken. Wer sie kennt, liest den Code als konsequent durchgehaltenen Plan.